

Task 1: test cases for your code

Function	Test case	Data/code	Does my code handle it? Here: handle= does NOT crash
sublist(link A, link pos_list)	Index out of bounds	A: 10 ->10 ->40 ->20 pos_list: <u>(-7)</u> -> 3 or pos_list: 3 -> <u>80000</u> -> 3 result: fct returns NULL	Yes for both cases
	A is NULL	link A = NULL; result: fct returns NULL	yes
	A is empty	link A = new_list(); result: fct returns NULL	yes
	pos_list is empty	link pos_list = NULL; result: fct returns NULL	yes
	pos_list is NULL	link pos_list = newList(); result: fct returns NULL	yes
	A is not modified by sublist(...)	A: 15 -> 100 -> 7 -> 5 -> 100 pos_list: 3 -> 0 ->2 result: A will still be : 15 -> 100 -> 7 -> 5 -> 100	yes
	Normal data (as in hw writeup)	A: 15 -> 100 -> 7 -> 5 -> 100 - > 7 -> 30 pos_list: 3 -> 0 -> 6 -> 4	yes
	Repeated position	A: 5 pos_list: 0 -> 0 -> 0 result: returns: 5-> 5-> 5	yes
delete_occurrences (link A, int V)	Normal data, V is in A (as in hw write-up)	A: 15 -> 100 -> 7 -> 5 -> 100 - > 7 -> 30 V is 7, Result: A will become: 15-> 100-> 5 -> 100 -> 30	yes
	V does not occur in A	A: 15 -> 100 -> 7 -> 5 V is 9, Result: A does not change: 15-> 100-> 7-> 5	yes
	Repeated consecutive occurrences	A: 15 -> 7 -> 7 -> 5 V is 7, Result: A becomes: 15 -> 5	yes
	A has one item and that is V	A: 7 V is 7 Result: A becomes Empty	yes
	A has only items with value V in it	A: 7->7-> 7 V is 7 Result: A becomes empty	no
	A is NULL	A = NULL Result: A is not changed	yes
	A is empty	A = new_list() Result: A is not changed	yes

swap_first_third(link A)	STUDENTS must give the special cases for this function. (Add or remove rows as needed.)	STUDENTS must give the example data	Does the code handle it?
	Normal Data	A: 15 -> 100 -> 7 -> 5 -> 100 -> 7 -> 30 Result : A will become 7 -> 100 -> 15 -> 5 -> 100 -> 7 -> 30	yes
	A is NULL	A = NULL Result: A is not changed	yes
	A is empty	A = new_list() Result: A is not changed	yes
	A has only 1 item	A: 10 Result: A becomes Empty	yes
	A has only 2 items	A: 10->20 Result: A will become 20->10	yes

Task 2 :

Let $N = |A|$ (N is the size of list A)

swap_first_third(link A): $T(N) = 1$

There are no loops, therefore the time complexity is constant.

delete_occurrences(link A, int V) $T(N) = N$

There is one while loop, and worst case is it will have to traverse through the whole list and iterate N times.

sublist(link A, link pos_list) $T(N) = N^2$

There is one while loop nested in another. The worst case for both is they will have to traverse through the

whole list so they will iterate $N*N$ times.

Task 3 (10 points) Given:

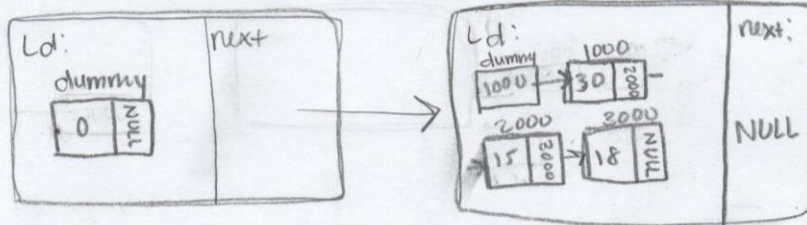
A new node structure (intended to be used to create a list of lists) is defined in the table below (using `struct node`):

<code>struct node {</code>	<code>struct coll_node {</code>
----------------------------	---------------------------------

<pre>int item; struct node * next; };</pre>	<pre>link Ld; // NOTE: Ld must be represented with a dummy starting node. struct coll_node * next; };</pre>
---	---

In your drawings, **show all the data as done in class** (including the list nodes, of type `struct node`). Use boxes for all member variables and write their value inside the box and their name outside the box.

a) (7 points) Draw two nodes (of type `struct coll_node`) that point to each other. For one of them `Ld` should be empty (but not NULL) and for the other one `Ld` should be: 30->15->18. Use the representation with a DUMMY node for any normal list, `Ld`, part of nodes of type `struct coll_node`.



b) (3 points) Assume that an `int` is stored in 4 Bytes and a memory address is 8 Bytes. How much space will the above two nodes (and the data that they reference) occupy? That is, give the total space needed to store in memory what you drew above. **SHOW YOUR WORK.**

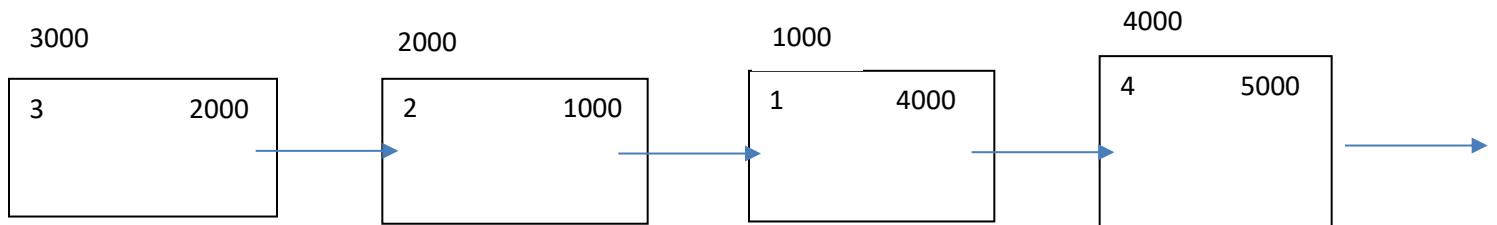
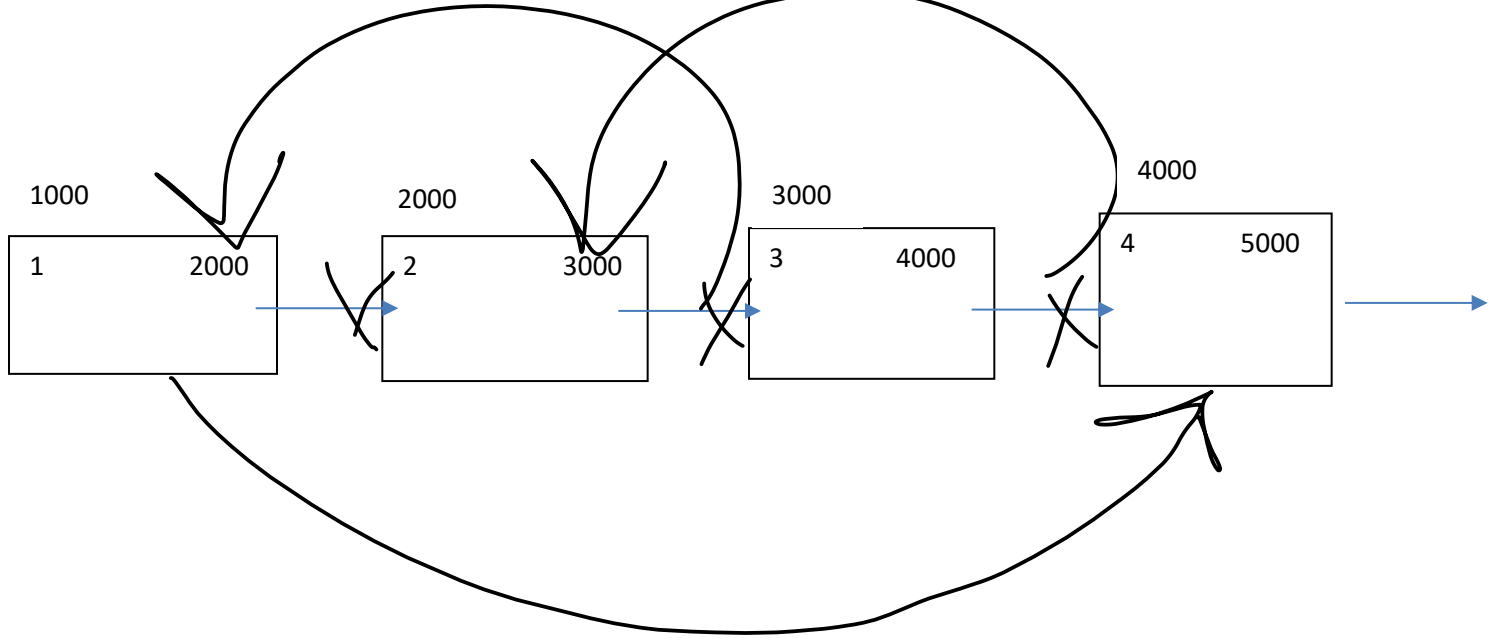
$$\# \text{ of ints} = 5 \cdot 4 = 20$$

$$\# \text{ of struct nodes} = 5 \cdot 8 = 40$$

$$\# \text{ of coll-nodes} = 2 \cdot 8 = 16$$

$$20 + 40 + 16 = 76 \text{ bytes}$$

Drawing of Swap First Third:



```
void swap_first_third(link A) {
    if(A == NULL || A->next == NULL || A->next->next == NULL)
        return;

    link Node = A->next;
    link Node2 = Node->next;
    if(Node2->next == NULL) {
        Node2->next = Node;
        Node->next = NULL;
        A->next = Node2;
        return;
    }

    link Node3 = Node2->next;
```

```
link temp = Node3->next;  
  
Node3->next = Node->next;  
  
Node->next = temp;  
  
A->next = Node3;  
  
Node2->next = Node;
```

```
    return;  
}
```

To Compile and Run Code:

```
gcc -g -o instr instructor_client.c list_hw.c  
./instr  
or  
valgrind --leak-check=full ./instr
```